

# Introduction to Unit Testing (1/2)

## pytest: Assertions, Fixtures, and Test Organization

SE 413/513 — Software Testing  
Grand Valley State University

# About This Class

- Today: Introduce the **idea of unit testing** and learn about pytest.
- We will focus on **pytest basics, assertions, exceptions, and test structure**.
- We will end with hands-on exercises so you can write and diagnose tests yourself.

© Dr. Kamrul Hasan, Grand Valley State University. All rights reserved.

# Learning Objectives

By the end of today, you should be able to:

- Define a unit test and explain the **FIRST** properties of a good unit test.
- Explain why pytest is useful and how it differs from unittest.
- Write and run basic pytest tests using plain assert statements.
- Interpret pytest assertion-failure output and use it to locate defects.
- Correctly test floating-point results and expected exceptions.
- Structure a test using the **Arrange–Act–Assert (AAA)** pattern.

# Roadmap

1 Introduction to Unit Testing

2 Why pytest?

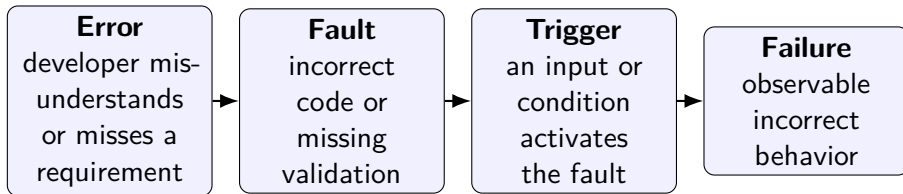
3 Assertions in pytest

4 Practice: Unit Testing with pytest

5 Recap

# From Software Failure to Unit Testing

A useful way to think about software testing is as a chain:



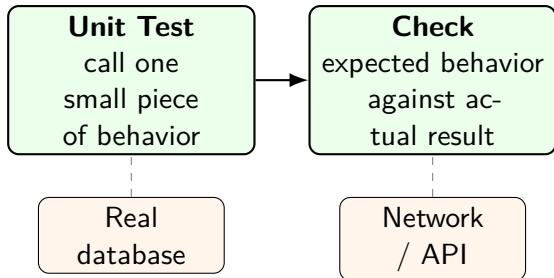
## Key Idea

**Unit testing helps us detect failures early by checking small pieces of behavior automatically.**

- A unit test gives us a precise, repeatable way to check whether a function behaves as required.
- When a unit test fails, the defect is narrowed to a small piece of code.
- The test can then remain in the suite as **regression protection**.

# What Is a Unit Test?

A **unit test** is a small, automated test that exercises *one isolated piece of behavior* in a program — typically a single function or method — and checks that it produces the correct result for a given input, without touching a real database, network, file system, or other parts of the application.



- **Small:** usually one function or method.
- **Isolated:** independent of external systems and other tests.
- **Automated:** the test determines whether the behavior passes or fails.

# What Makes a Good Unit Test? — FIRST

A useful unit test follows the **FIRST** principles:

| Property             | Meaning                                                                                                                     |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>Fast</b>          | Runs quickly, so developers can run tests frequently during development.                                                    |
| <b>Isolated</b>      | Tests one thing and does not depend on other tests or their execution order.                                                |
| <b>Repeatable</b>    | Produces the same result in any environment; no dependence on network access, the system clock, or uncontrolled randomness. |
| <b>Self-checking</b> | The test automatically reports pass or fail; a human does not have to inspect the output.                                   |
| <b>Timely</b>        | Written close to the same time as the code it tests, rather than being added as an afterthought.                            |

# Roadmap

1 Introduction to Unit Testing

2 Why pytest?

3 Assertions in pytest

4 Practice: Unit Testing with pytest

5 Recap

## The Chain in Code: Setting Up the Fault (Last Week)

A developer misunderstands a billing requirement (the *error*): they believe a late fee can never be negative, so they never considered a negative `days_late`.

```
def calculate_late_fee(days_late, daily_rate=1.50, max_fee=25.00):  
    """Return the late fee for an overdue library book."""  
    fee = days_late * daily_rate  
    if fee > max_fee:  
        return max_fee  
    return fee
```

```
>>> calculate_late_fee(3)  
4.5  
>>> calculate_late_fee(30)  
25.0  
>>> calculate_late_fee(-4)  
-6.0
```

The fault (missing input validation) was present the whole time; it just needed the right **triggering condition** (a negative `days_late`) to produce an observable failure.

# A Unit Test, Revisited (Last Week)

Recall `calculate_late_fee()` from Lecture 1. A minimal unit test, using Python's `unittest`:

```
class TestLateFee(unittest.TestCase):
    def test_typical_fee(self):
        self.assertEqual(calculate_late_fee(3), 4.5)

    def test_negative_days_should_not_be_negative_fee(self):
        # Documents the requirement; currently FAILS, exposing
        # the fault from Lecture 1.
        self.assertGreaterEqual(calculate_late_fee(-4), 0)
```

## Tip

A failing test that accurately captures a requirement is not a problem to hide — it is exactly what Principle 1 describes: it shows the presence of a defect.

# Why Not Just unittest?

Python ships with a built-in framework, `unittest`, which you've already used: tests are methods on a class inheriting from `unittest.TestCase`, and comparisons go through a large family of methods (`assertEqual`, `assertTrue`, `assertIn`, ...).

`pytest` is a third-party framework that has become the de facto standard for testing Python code, in industry and open source alike.

```
pip install pytest
```

Two design decisions explain most of its popularity — and both are the spine of today's lecture: **(1) plain assert statements** and **(2) fixtures**.

# Test Discovery and Running Tests

pytest finds tests automatically, with no configuration, by following naming conventions:

- Test files: `test_*.py` or `*_test.py`.
- Test functions inside them: `test_*`.
- Test classes: `Test*` (no `__init__`); test methods still `test_*`.

```
pytest                # discover and run every test
pytest test_pricing.py  # run everything in one file
pytest test_pricing.py::test_addition  # run one test
pytest -v              # verbose: list each test by name
pytest -k "addition"   # run only tests matching a name
```

## Tip

Run pytest constantly — ideally on every save, always before committing. This is what turns the *shift-left testing* idea from the *Testing Foundations* reading into a daily habit.

# Roadmap

- 1 Introduction to Unit Testing
- 2 Why pytest?
- 3 Assertions in pytest**
- 4 Practice: Unit Testing with pytest
- 5 Recap

# A Test Is Just a Function

A pytest test is, at its core, a Python function that raises no exception if everything is fine, and raises `AssertionError` if something is wrong. The simplest test body is one `assert`.

```
def calculate_discounted_price(price, discount_percent):  
    return price * (1 - discount_percent / 100)  
  
def test_ten_percent_discount():  
    assert calculate_discounted_price(100, 10) == 90
```

No special base class, no special method name pattern beyond `test_*` — just a function and an `assert`.

# The Payoff: Assertion Rewriting

pytest rewrites your test files' bytecode at collection time so a *failing* assert reports exactly what was compared — with no special assertion method required. Suppose the function above has a fault:

```
def calculate_discounted_price(price, discount_percent):  
    return price * (1 - discount_percent) # BUG: forgot / 100
```

```
def test_ten_percent_discount():  
>     assert calculate_discounted_price(100, 10) == 90  
E     assert -900.0 == 90  
E       + where -900.0 = calculate_discounted_price(100, 10)
```

## Key Idea

One plain assert yields the actual value, the expected value, *and* which call produced it. In unittest, assertTrue(...) alone would only report “False is not true” — no values shown.

# Comparing Floating-Point Numbers

Direct float equality is risky, because of how floating-point arithmetic works:

```
>>> 0.1 + 0.2 == 0.3  
False
```

`pytest.approx()` handles this correctly:

```
import pytest  
  
def test_float_addition():  
    assert 0.1 + 0.2 == pytest.approx(0.3)
```

## Common Pitfall

An assertion with no diagnostic value, like a bare `assert result`, tells you a test failed but not why. Prefer `assert result == expected` so assertion rewriting can show you both sides.

## Testing for Exceptions: `pytest.raises()`

Some behavior is only correct if it raises the right exception. `pytest.raises()` is a context manager asserting that a specific exception type is raised inside its block.

```
import pytest

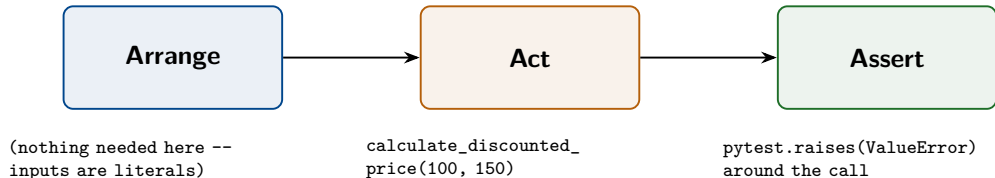
def calculate_discounted_price(price, discount_percent):
    if not 0 <= discount_percent <= 100:
        raise ValueError("discount_percent must be between 0 and 100")
    return price * (1 - discount_percent / 100)

def test_invalid_discount_raises_value_error():
    with pytest.raises(ValueError):
        calculate_discounted_price(100, 150)
```

If the block raises nothing, or the wrong exception, the test fails — exactly the guarantee you want for error-handling paths.

# Arrange–Act–Assert (AAA)

Well-organized tests, in any framework, tend to share the same three-part shape. Applied to the test on the previous slide:



## Key Idea

**Arrange** the inputs and any needed state, **Act** by calling the code under test, **Assert** the outcome is correct. Being able to spot these three parts at a glance is a good sign a test is doing one clear thing.

# Roadmap

- 1 Introduction to Unit Testing
- 2 Why pytest?
- 3 Assertions in pytest
- 4 Practice: Unit Testing with pytest**
- 5 Recap

## Exercise: Is This a Good Unit Test?

For each proposed test, decide whether it is a good unit test. Identify which FIRST property is most clearly violated, if any, and explain why.

- A test calls a function that sends a real HTTP request to a public website.
- A test depends on the result of another test having run first.
- A test checks a calculation using an exact floating-point equality.
- A test has a clear assertion and uses only local, deterministic inputs.

**Try it yourself before discussing with a partner.**

## Exercise: Write Your First pytest Test

Write a pytest test for the following function. Use a plain assert and choose an input that gives you an easy expected value.

```
def calculate_total(price, quantity):  
    return price * quantity
```

*# Write your test below.*

**Try it yourself before discussing with a partner.**

## Exercise: Repair the Float Test

The following test can fail even though the mathematical result is correct. Rewrite it so that it is an appropriate pytest test.

```
def test_total():  
    result = 0.1 + 0.2  
    assert result == 0.3
```

**Try it yourself before discussing with a partner.**

## Exercise: Test an Error Path

The function should reject a negative quantity. Write a test that passes only when the function raises `ValueError`.

```
def make_purchase(price, quantity):  
    if quantity < 0:  
        raise ValueError("quantity cannot be negative")  
    return price * quantity
```

*# Write your test below.*

**Try it yourself before discussing with a partner.**

## Exercise: Refactor into AAA

Take the following test and identify its **Arrange**, **Act**, and **Assert** parts. Then rewrite it so the three stages are visually obvious.

```
def test_discount():  
    assert calculate_discounted_price(100, 10) == 90
```

**Try it yourself before discussing with a partner.**

# Roadmap

- 1 Introduction to Unit Testing
- 2 Why pytest?
- 3 Assertions in pytest
- 4 Practice: Unit Testing with pytest
- 5 Recap**

# Class 1: Key Takeaways

- A unit test checks one small, isolated piece of behavior.
- Good unit tests are **FIRST**: Fast, Isolated, Repeatable, Self-checking, and Timely.
- `pytest` lets us use plain `assert` statements and gives useful failure diagnostics.
- Use `pytest.approx()` for floating-point comparisons and `pytest.raises()` for expected exceptions.
- Use **Arrange–Act–Assert** to make tests easy to read and reason about.

## Key Idea

Next class: how fixtures, scopes, parametrization, and `conftest.py` make these tests scalable.

# Questions?

Thank you.

SE 413/513 — Software Testing  
Grand Valley State University